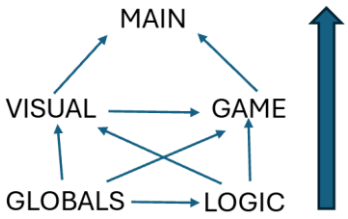
Programozói dokumentáció

A program felépítése

A program a main fájlon kívül 2 fő, illetve 2 segéd modulból áll, ezek a main.c, visual.c, game.c, logic.c illetve a globals.c fájlok. A c fileok a “scr” mappában míg az ezekhez tartozó fejléc fileok az “Include” mappában találhatóak.

A modulok alulról (segéd modulok) középre (fő modulok) fentre (main modul) függenek egymástól, így elkerülve a körkörös függést.



A program megjelenítése egy külső könyvtár segítségével történik, ez a bonyolódások elkerülése miatt megtalálható lokálisan az “Ncurses” mappában.

A játék állását egy .game bináris fájlba menti, illetve onnan tölti be a játék, ami a program .exe mappájában kell lennie.

Jobb átláthatóság miatt a modulok bemutatása alulról kezdődik, hogy később minden érthető legyen.

Compile parancs: gcc src/*.c ncurses/pdcurses/*.c ncurses/wincon/*.c -Iinclude -Werror -Wall -o JOTW.exe

Logic module – Logikai modul

Ebben a modulban alapvető több felhasználással rendelkező függvények találhatóak.

max és min

Két egész szám közül adja vissza a kisebbet, illetve a nagyobbat, itt csak egész számok jöhetnek szóba, ezért nem az fmax és fmin függvények vannak használatban a math.h standard könyvtárból.

randint

Két egész szám között ad vissza egy random generált random egész számot beleértve mindkét bejövő számot is.

calc_chance

Bemenete egy egész szám, ami 0 és 100 között egy százalék esélyt jelent. A függvény generál a randint segítségével egy 0 és 100 közötti egész számot és visszaadja, hogy a generált szám kisebb-e az esélynél. Így egy bool értéket kapunk arról, hogy “bejött-e” az adott esély.

format_string

Bemenete tetszőleges számú, minimum 1, logikusan minimum 2. Első bemenete egy formázott sztring majd a többi bemenet a sztringbe kívánt értékek, a függvény feladata egy dinamikus hosszúságú sztring létrehozása, ahol az értékek már a helyükre kerültek. A függvény működése a következő, a stdarg.h standard könyvtár segítségével bekérjük a formátumot és a tetszőleges mennyiségű értéket. Az eredmény hosszúsága hiányában először megpróbáljuk létrehozni a kész sztringet a beépített vsnprintf segítségével, itt eltároljuk a hosszt, amit visszaad. Amennyiben rossz formátumot adtunk volna meg itt megszakad a program a 2-es hibakóddal (formátum hiba). Ezután lefoglal egy kellő hosszúságú memória területet az eredménynek. Itt 1-es hibakód (foglalás hiba), ha nem sikerül a memóriafoglalás. Ezek után mégegyszer lefut a vsnprintf és beírja a kimenetbe a teljes sztringet. A memória foglalását a megjelenítés után szabadítjuk fel.



Bar_string

A format_string-hez hasónlóan működő függvény, ami egy üzenetből és a bemenő 2 egész számból (current_value és max_value) létrehoz egy olyan dinamikus hosszúságú sztringet ami igy néz ki: <message> [####------]. Az elején megjelenik az üzenet majd a

2 szám alapján kitölt egy 10 karakter hosszú “dobozt” “#” illetve “-” karakterekkel. A karakterek mennyisége attól függ, hogy a current_value hány százaléka a max_value-nek tízesekre kerekítve. Ezt a kimenő sztringet is futás közben foglaljuk le majd megjelenítésnél szabadítjuk fel.



Save_game – Load_game

A játék állását menti, illetve töltik be, egy .game bináris fájlból. Először a game strukturát teljes egészében olvassa be vagy írja ki majd a playerben található láncolt lista elemeit olvassa be vagy írja ki és újra felépíti a láncolt listát.

Betöltésnél használatban van még a save_exists függvény ami ellenőrzi a mentési fájl meglétét.

Globals module - Globális modul

A Globals modul tartalmaz minden struktúra definíciót, globális változót és előre létrehozott struktúrákat, amire más moduloknak szüksége van.

Globalis valtozók

A játék többféle előre megadott adattal dolgozik, ilyenek a

- Játék nehézségét vezérlő adatok
- Játék szintjeinek adatai
- Játékos és az ellenség konstansai
- Színkódok a megjelenítéshez
- Tulajdonságkódok a játékos adataihoz

Struktúrák

- Game:

Egy játékállás tárolása, tartalmazza a Játékos struktúráját, az épp jelen lévő ellenség struktúráját. Számon tartja az éppen következő felet és a nehézségi mutatót is.

- Player

Egy játékos leírása, tartalmazza minden adatát egy játékosnak. Többek között név, életerő, képességet sebzése és még sok más

- Enemy

Szintén tartalmazza egy ellenség összes adatát

- Item, ItemBase és Slot

A Slot annak a láncolt listának eleme, amit a player struktúrában a tárgyakat tárolja. Kezdő mutatót a player struktúrában tároljuk. Az Item és az ItemBase a tárgyakért felelnek, az ItemBaseből generálódik le az Item random generált adatokkat az előre megadott max és min érték között.

Visual module - megjelenítő modul

A visual modul felel az összes képernyőn megjelenő vizuális elemért. Az Ncurses könyvtárat használva valósul meg.

Init_screen

Itt kezdődik meg a megjelenítés felépítése. Megállapítjuk a képerníő méretét, beállításra kerülnek a színek.

Screen tools

A line, side és middle függvények a megjelenítés fő alapelemei. A line a bemenő sorra ír egy vonalat. A side illetve a middle az adott sorra a bemenő akár statikus akár dinamikus sztinget írják ki vagy két oldalra vagy középre, a side-nak épp ezért két sztring bemenete van. Szükségük van ezen kívül egy írasi színre, ezeket az init_screen-ben majd a globalsban definiáltuk, valamint egy flagre amit megadja, hogy dinamikus volt-e a sztring, ekkor felszabadítja őket kíirás után.

Található még két univerzális rajzoló függvény, amik közül valamelyik minden másik Rajzoló függvény elött aktiválódik és megrajzolja az alap vázát a kijelző tartalmának.

Wrappers

Minden draw függvény egy wrapperben fut le, itt először letöröljük a képernyőt a rajzoló függvénnyel rajzolunk majd frissítjük a kijelzőt.

Menu_logics

Minden menü léptetési logikája külön található kisebb változtatásokkal. A fő működési elv viszont ugyanaz, létrehoz egy változót, amit növel vagy csökkent a lenyomott gomb alapján majd ezt adja tovább a rajzoló függvénynek, ott majd ez alapján lehet rajzolni különböző állapotokat. Például a kiválasztott elem más színű mint a többi.

Game module - Játék logika modul

A game modulban található az összes funkció, ami a játékmenethez kellhet. Minden függvény egy game struktúra pointert vesz be és az alapján végzi el a feladatát. A modul globális függvényei a “create_new_game” illetve a “game_loop” amiket a main használ.

scale_enemy

A nehézség alapján megnöveli az épp jelen lévő ellenség adatait. Akkor van használatban amikor egy új ellenséget generálunk

generate_enemy

A híváskori stage és enemy_index alapján lemásol egy ellenség struktúrát a game-be és felerősíti azt a nehézség alapján (scale enemy).

create_new_game

Meghívja a nehézség választó menüt és annak eredménye alapján visszaad egy új game struktúrát alapértelmezett player sztuktúrával és egy az adott szintből random generált ellenséggel, illetve egy random generált turn változóval is, ami eldönti, hogy melyik fél fog kezdeni.

genetrate_item

Egy random generált szerencse szám alapján választ egy ritkasági szintet és abban egy Item_base-t a global modulban megadottak közül, majd a boost_min és a boost_max között generál egy értéket, ami az új Item struktúra adata lesz, valamint minden más adatot átmásol az új Item-be. Így sok tárgy esetén memória spórolás miatt csak azokat az adatokat tároljuk el, amikre biztosan szükség van.

add_item

Az adott tárgyat hozzáadja a játékos láncolt listájához.

Képességek

A zap, charge, meteor, player_heal és player_rest mind a player_turn eredménye alapján kiválasztott lépés, amit a játékos végrehajt. Az első három levonja manát, amibe kerülnek, miss chance alapján nem találnak, crit chance alapján kritikusak, az ellenfél armor és crit_dmg alapján sebzést számolnak. A zap különleges képessége, hogy sima találatnál adott eséllyel lebénítja az ellenfelet, kritikus találatnál biztosan. A meteor sok manába kerül cserébe sokat sebez illetve felgyújtja az ellenfelet ami akár több körön át égei azt. A charge nem támadásra, hanem saját képességek felerősítésére használható. Használatkor szintén tud kritikus lenni ekkor többet ad hozzá a current_charge-hoz ami a következő képességhasználatkor a charge_mult

alapján több sebzést és esetleg tüzet fog okozni. Az utóbbi 2 lépés lehetőség a gyógyulás és a pihenés, elöbbivel amiennyiben rendelkezik a játékos gyógyitallal gyógyulni, az utóbbival pedig mana visszaszerzésére van lehetőség.

enemy_turn

Itt egy rövid döntési háló található az ellenség lépéseiről, amennyiben az életereje enemy_heal_threshold alatt van gyógyul, egyéb esetben pedig támad.

enemy_status

Az ellenség tűz értéke alapján sebzést okoz az ellenfélnek. Később bővíthető.

enemy_death

Az ellenség halálakor a tapasztalati pontot a játékosnak adja és amennyiben szintlépés történt, generál három tárgyat. A game_item menü eredménye alapján az egyiket az Inventoryba rakja.

player_death

Leellenörzi a játékos életerejét és amennyiben 0 a játék véget ér, meghívja a game_over menüt, ami után visszatérünk a főmenübe.

game_loop

A játék ciklus-t tartalmazza, használja a legtöbb feljebb megemlített függvényt. A ciklus végén a free_inventory függvénnyel felszabadítja a láncolt lista elemeit.

A main modul

Végül a main modul a program indításakor, először inicializálja a megjelenítés. Létrehoz egy Game struktúrát és elindítja a program ciklust, amiben a main_menu-t hívja meg és annak visszatérési értéke alapján vagy új játékot kezd, betölt egy meglévő játékot vagy kilép a quit menu segítségével. Amennyiben új vagy meglévő játékra kerül a sor, létrehozza vagy betölti azt és elindítja a játék ciklust az adott játékkal. Emiatt amikor véget ér egy játék ciklus akkor visszatérünk a főmenübe.